

Implicplot v0.2.0

Uwni 鶯鴒

Abstract

The module provides two entry points, which answer different questions about a relation written as one string, comparison included: $\sin(x^2 + y^2) = \cos(x*y)$. `plot` determines **which pixels the relation may touch** and returns a raster of one byte per pixel, computed by adaptive subdivision; it is rigorous and can fill an inequality. It implements the reliable-graphing method of Tupper [1]. `contour` determines **where the curve is** and returns polylines with a guaranteed topology, by the subdivision approach of Plantinga & Vegter [2] with the stopping rule of Lin & Yap [3]. Neither returns an image: both return data, and the document decides how to render it.

Each figure below is shown next to its source.¹

What a relation may say

A relation is two expressions with a comparison between them — one of `=`, `<`, `<=`, `>`, `>=`, where `==` is also accepted for `=`. Which side is which does not matter: everything is reduced to $f \text{ op } 0$ before it is plotted, and the subtraction of a literal zero is dropped, so `"f > 1"` and `"f - 1 > 0"` are not merely equivalent but the same program.

An expression is built from x and y , the constants `pi`, `tau` and `e`, decimal literals (`0.5`, `.5` and `1.5e3` all read), the operators `+` `-` `*` `/` `%` with a leading `-` for negation, parentheses, and `^`. Whitespace is insignificant, and the dedicated minus sign — (U+2212) — what math mode renders, so it survives copying a formula out of a document — is accepted wherever `-` is, `1e-5`-style axis labels included. The functions are

```
abs neg sin cos tan exp sqrt log asin acos atan floor ceil
min(a, b) max(a, b)
```

with `ln`, `arcsin`, `arccos` and `arctan` as alternative spellings of four of them. That is the whole list, and it is not really a list: a name is looked up directly in the evaluator's operator enum, so the language has exactly the operations the two arithmetic domains implement and cannot come to disagree with them.

`%` is floored modulo at the precedence of `*` and `/`, taking the sign of its divisor as Python's `%` does, so `7 % -3` is `-2`. `floor`, `ceil` and `%` are the only operations here that are not continuous, and the step is not swept under the carpet: a cell whose image crosses one is marked discontinuous in the arithmetic itself, `contour` refuses to trace through it and counts it in `uncertain`, and `plot` — which needs only the enclosure, never continuity — is unaffected. Expect `contour("x % 3 = 1")` to report a positive `uncertain`; the vertical lines it does return are still correct.

Two restrictions are worth knowing before they surprise you. There is **no implicit multiplication** — `2x` and `x y` are errors, and `mean 2*x` and `x*y`. And `^` takes an **integer literal** only: `x^2` and `e^-1` are fine, `x^y` and `x^0.5` are not. The restriction is not laziness but the reason the plotter converges: a

¹The snippets assume `#import "lib.typ" as ip` and four drawing helpers from `manual-helpers.typ`, next to this file: `art` (renders a raster as ASCII), `stroke-chains` (draws polylines into a box), `fill-chains` (fills closed chains as one even-odd vector path), and `fill-runs` (draws a raster as run-length rects). Each is about fifteen lines of plain Typst. The core of `stroke-chains` is:

```
place(curve(
  curve.move(points.first()),
  ..points.slice(1).map(p => curve.line(p)),
))
```

power is a single operation that knows its argument occurs once, enclosing $[-1, 1]^2$ as $[0, 1]$, where $x \cdot x$ must assume the two factors are independent and can only manage $[-1, 1]$. Use `sqrt` for a square root and `exp(y * log(x))` if a variable exponent is really meant.

Anything else is refused with the offending character marked, which Typst reports at the call site:

error: plugin errored with: unknown name

```
sin(x^2 + y^2) = cos(x*z)
```

^

plot: one byte per pixel

The relation $x < 0$ over $[-1, 1]^2$ at 8×8 returns 64 bytes, row-major from the top:

```
#raw(ip.rows(
  ip.plot("x < 0", size: (8, 8),
    x: (-1, 1), y: (-1, 1)),
  8).map(row => row.map(str).join("
")).join("\n"))
```

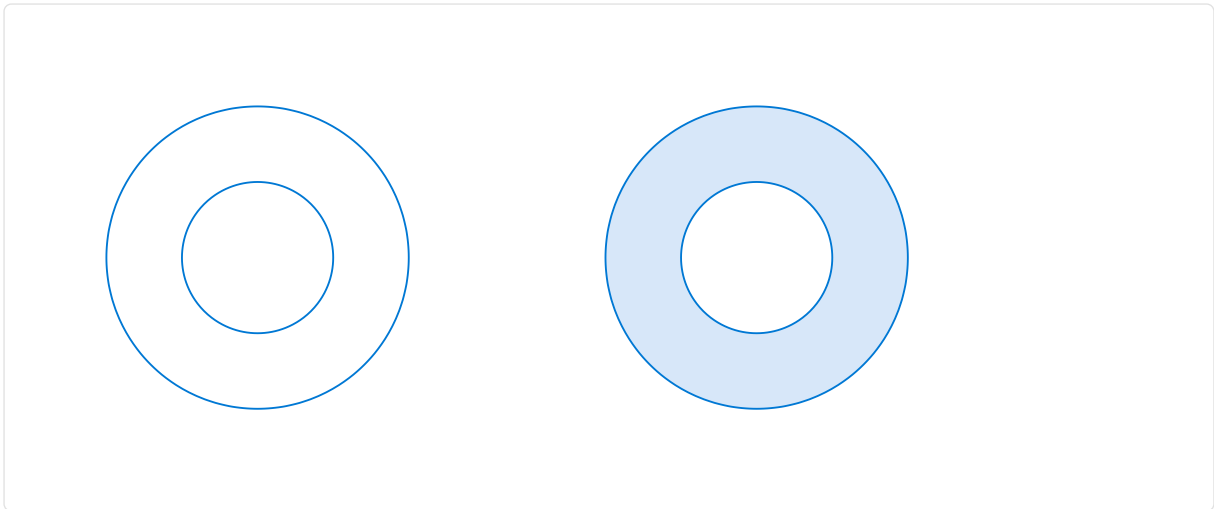
```
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
```

An inequality yields a filled region, which is what the raster represents:

```
#art(ip.plot("sin(x^2 - y^2) < sin(x + y) + cos(x*y)",
  size: (76, 30), x: (-4, 4), y: (-2, 2)), 76)
```

contour: the curve as polylines

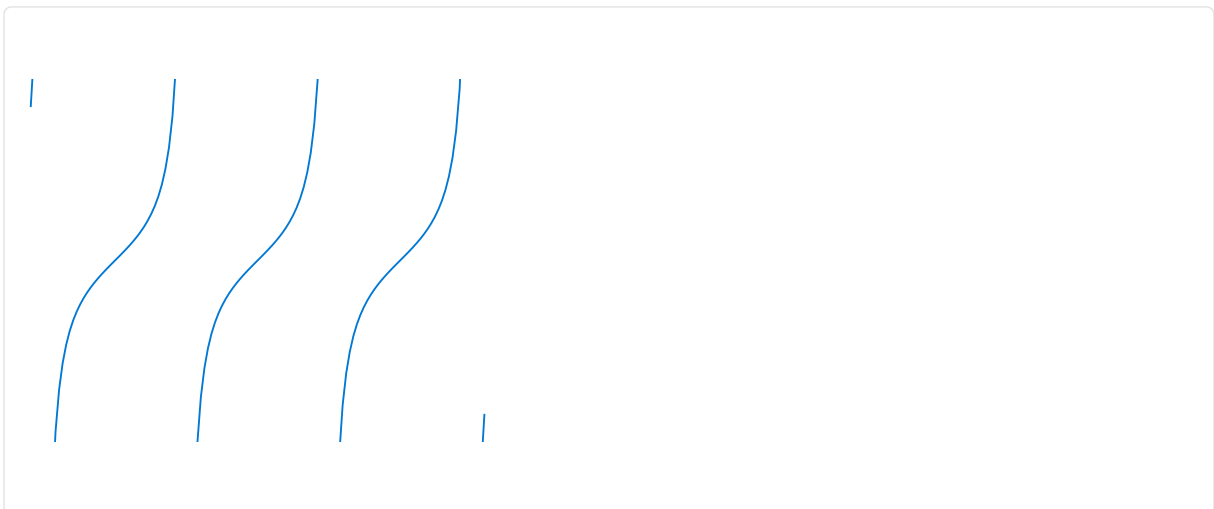
```
#let rings = "(x^2 + y^2 - 4) * (x^2 + y^2 - 1) = 0"
#let circles = ip.contour(rings, n: 80, x: (-3, 3), y: (-3, 3))
#grid(columns: 2, gutter: 0.6cm,
  box(width: 6cm, height: 6cm,
    stroke-chains(circles.chains, (-3, 3), 6cm)),
  box(width: 6cm, height: 6cm,
    fill-chains(circles.chains, (-3, 3), 6cm, stroke: 0.7pt + accent)),
)
```



Poles and discontinuities

A sampling tracer reads a sign change between two samples as a crossing. At an asymptote that sign change is a discontinuity rather than a root, and interpolating across it produces a segment that is not part of the curve. `contour` evaluates every cell in decorated interval arithmetic (IEEE 1788 [4]) and rejects cells across which the relation is not continuous, so $y = \tan(x)$ produces no segment at its asymptotes:

```
#let tangent = ip.contour("y = tan(x)", n: 90, x: (-5, 5), y: (-4, 4))
#box(width: 6cm, height: 6cm,
  stroke-chains(tangent.chains, (-5, 5), 6cm))
```



Each of the 510 returned points lies within one cell of the true curve; the build asserts this. The pole columns are still counted: uncertain is 2048, because nothing about those cells was certified. They contribute no geometry.

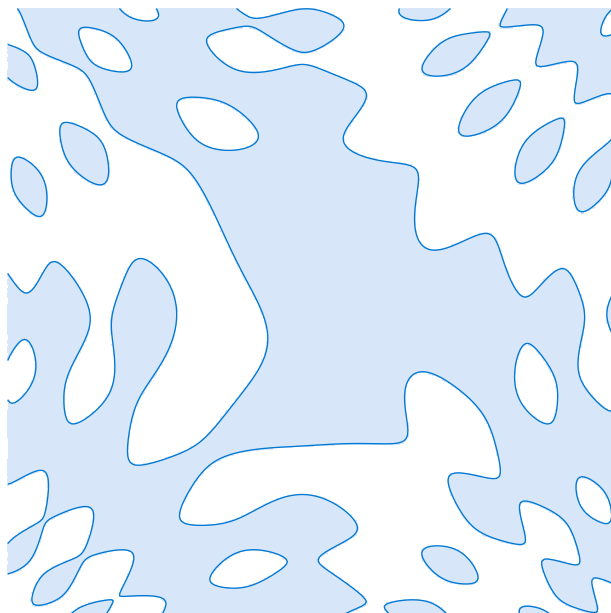
Combining fill and stroke

The two entry points answer different questions about the same relation and can be combined in one figure: `plot` fills the inequality — its covering guarantee means no part of the region is omitted — and `contour` strokes the boundary, which is the zero set of the corresponding equality and stays sharp at any zoom:

```

#let g = "sin(x^2 - y^2) < sin(x + y) + cos(x*y)"
#let g-fill = ip.plot(g, size: (500, 500), x: (-4, 4), y: (-4, 4))
#let g-edge = ip.contour(g, n: 150, x: (-4, 4), y: (-4, 4))
#box(width: 8cm, height: 8cm, {
  fill-runs(g-fill, 500, 8cm)
  stroke-chains(g-edge.chains, (-4, 4), 8cm, weight: 0.5pt)
})

```



The two results agree because both are handed the same string: `plot` reads its comparison and fills $f < 0$, `contour` ignores the comparison and traces the boundary $f = 0$. Where that boundary has singular points, `contour` reports them rather than guessing — 0 cells here.

This figure uses a raster fill rather than the vector fill used for the annulus, because the region leaves the window on all four sides and most of its boundary chains are therefore open. Filling an open path closes it with a straight chord and paints the wrong region; closing chains correctly along the window border is geometry the document would have to implement itself. The raster answers the region query directly and errs outward, never inward.

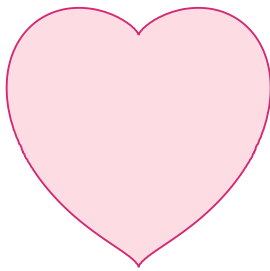
Gallery

```

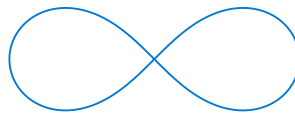
#let heart-edge = ip.contour("(x^2 + y^2 - 1)^3 <= x^2*y^3",
  n: 100, x: (-1.5, 1.5), y: (-1.4, 1.6))
#let lemniscate-edge = ip.contour("(x^2 + y^2)^2 = x^2 - y^2",
  n: 80, x: (-1.2, 1.2), y: (-1.2, 1.2))
#let folium-edge = ip.contour("(x^2 + y^2)^2 = x^3 - 3*x*y^2",
  n: 80, x: (-1, 1.2), y: (-1.1, 1.1))
#let pink = rgb("#d42873")
#grid(columns: (4.6cm,) * 3, gutter: 0.5cm,
  [
    #box(width: 4.6cm, height: 4.6cm, fill-chains(
      heart-edge.chains, (-1.5, 1.5), 4.6cm,
      fill: color.mix((pink, 18%), (white, 82%)),
      stroke: 0.7pt + pink,
    ))
    $(x^2 + y^2 - 1)^3 <= x^2 y^3$ \
    #chains-label(heart-edge.chains.len()), uncertain #heart-edge.uncertain
  ],

```

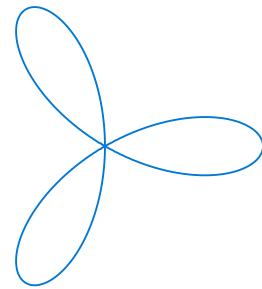
```
[
  #box(width: 4.6cm, height: 4.6cm,
    stroke-chains(lemniscate-edge.chains, (-1.2, 1.2), 4.6cm))
  $(x^2 + y^2)^2 = x^2 - y^2$ \
  #chains-label(lemniscate-edge.chains.len()), uncertain #lemniscate-edge.uncertain
],
[
  #box(width: 4.6cm, height: 4.6cm,
    stroke-chains(folium-edge.chains, (-1.1, 1.1), 4.6cm))
  $(x^2 + y^2)^2 = x^3 - 3x y^2$ \
  #chains-label(folium-edge.chains.len()), uncertain #folium-edge.uncertain
],
)
```



$(x^2 + y^2 - 1)^3 \leq x^2 y^3$
1 chain, uncertain 8



$(x^2 + y^2)^2 = x^2 - y^2$
2 chains, uncertain 4

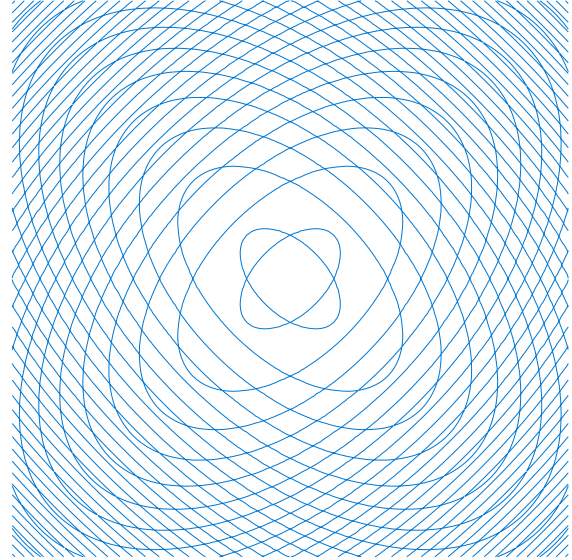
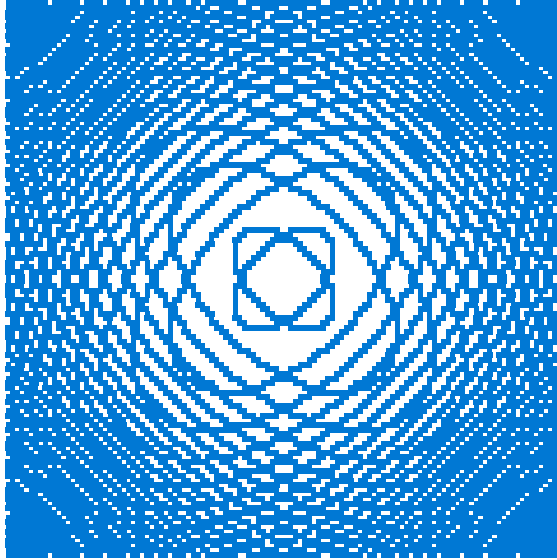


$(x^2 + y^2)^2 = x^3 - 3xy^2$
2 chains, uncertain 6

A non-zero uncertain counts cells in which the curve is singular — the lemniscate's self-intersection, the heart's two pinches — where the gradient vanishes and no amount of subdivision can settle the topology. The curve is still drawn; the count marks where the topology is not certified. The heart uses the vector fill, which is exact to its polyline, so the fill is only as reliable as the boundary's topology.

Guarantees and their limits

```
#let f = "sin(x^2 + y^2) = cos(x*y)"
#let pixels = ip.plot(f, size: (120, 120), x: (-8, 8), y: (-8, 8))
#let traced = ip.contour(f, n: 120, x: (-8, 8), y: (-8, 8), refine: 4)
#grid(columns: 2, gutter: 0.6cm,
  box(width: 7.5cm, height: 7.5cm,
    fill-runs(pixels, 120, 7.5cm, color: accent)),
  box(width: 7.5cm, height: 7.5cm,
    stroke-chains(traced.chains, (-8, 8), 7.5cm, weight: 0.35pt)),
)
```



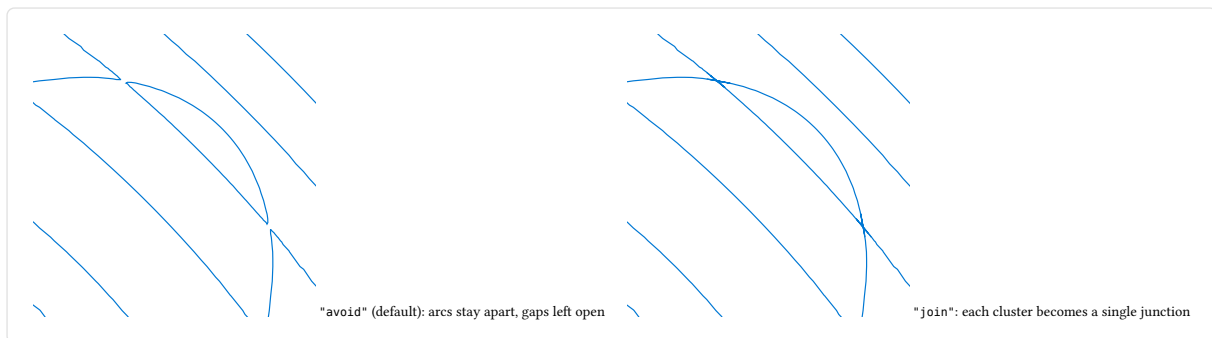
Both entry points subdivide adaptively; they differ in what they guarantee. The raster guarantees **coverage**: every true solution lies under a lit pixel, at any resolution. The polylines guarantee **topology**, on the basis of Snyder's parameterizability criterion [5] as adopted by [3]. Here $\sin(x^2 + y^2)$ oscillates faster than the cells at this n , and contour reports it: **8240** cells are unresolved, so the topology of this figure is not certified even though the rendering looks clean. For the circles above the count is zero and the shape is guaranteed; for the tangent, the 2048 cells are its pole columns, counted but drawn as nothing.

Uncertain cells: avoid and join

Whether the crossings are real is a property of the expression, not of the tracer. Here $\sin A - \cos B = 2 \cos\left(\frac{A-B}{2} + \frac{\pi}{4}\right) \cdot \sin\left(\frac{A+B}{2} - \frac{\pi}{4}\right)$, so the zero set is the **union** of two ellipse families — $x^2 - xy + y^2 = c$ and $x^2 + xy + y^2 = c'$ — and every meeting of the two families is a genuine intersection. At such a point the gradient vanishes, and no subdivision method can certify how the arcs connect; those are exactly the cells uncertain counts. The curve is defined there; what cannot be computed is the connectivity.

The connectivity is therefore the caller's choice. The default, uncertain: "avoid", keeps the arcs apart, which is the correct reading when a crossing is in fact a near miss. Here the factorisation shows otherwise, so uncertain: "join" contracts each cluster of uncertain cells to a single junction and the arcs meet in an X:

```
#let avoid = ip.contour(f, n: 120, x: (-8, 8), y: (-8, 8))
#let join = ip.contour(f, n: 120, x: (-8, 8), y: (-8, 8),
  uncertain: "join")
#grid(columns: 2, gutter: 0.6cm,
[
  #box(width: 7.5cm, height: 7.5cm, clip: true,
    stroke-chains(avoid.chains, (1.2, 3.6), 7.5cm, weight: 0.9pt))
  `avoid` (default): arcs stay apart, gaps left open
],
[
  #box(width: 7.5cm, height: 7.5cm, clip: true,
    stroke-chains(join.chains, (1.2, 3.6), 7.5cm, weight: 0.9pt))
  `join`: each cluster becomes a single junction
],
)
```



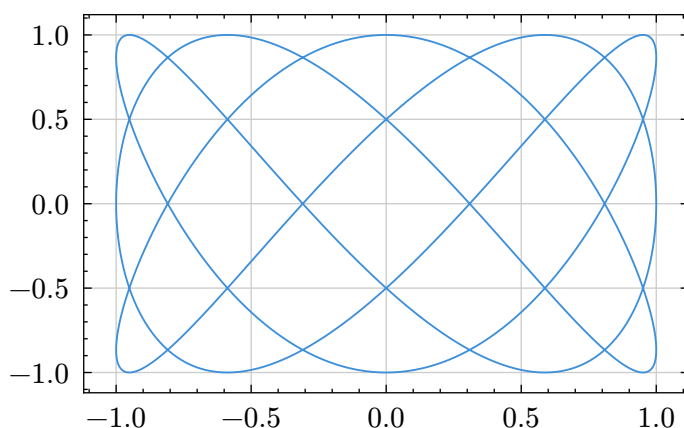
Both panels show the same page-scale trace as the figure above, zoomed to the detail window $[1.2, 3.6]^2$. They are not retraced at that window: retracing would deepen the tree, shrink the uncertain clusters and close the gaps almost regardless of the policy. The policy determines the connectivity at whatever scale the trace was made. The choice cannot be automated, because deciding whether an elementary expression is exactly zero at a point is undecidable in general [6], and near misses and true crossings are indistinguishable to interval subdivision — which is what uncertain reports. It can, however, be made on the basis of the relation: use "join" when the relation factors, as it does here, and "avoid" when the branches merely approach each other.

Writing relations efficiently

The numbers quoted below are computed during this build, so they describe the document you are reading.

If a parametric form exists, use it directly; this plugin is NOT needed. An implicit plotter is required only when the relation cannot be solved for a parameter. A Lissajous figure is awkward as an implicit equation and straightforward as a parameter sweep: every sampled point lies exactly on the curve, nothing is uncertain, and any plotting library can draw it:

```
#let ts = range(721).map(k => 2 * calc.pi * k / 720)
#lq.diagram(
  width: 8cm, height: 5cm,
  lq.plot(ts.map(t => calc.cos(3 * t)), ts.map(t => calc.sin(5 * t)),
    mark: none),
)
```



Clear denominators. The guarantees assume the relation is continuously differentiable. A pole falls outside that model; it is handled soundly, but at a cost. $y = \tan(x)$ traces with 2048 uncertain cells, which are its pole columns. Written as $\sin(x) = y * \cos(x)$ the relation is analytic and has

the identical zero set, and it traces with **0** uncertain cells — the full guarantee, from a one-line rewrite.

Write powers as powers. x^2 encloses $[-1, 1]^2$ as $[0, 1]$, whereas $x*x$ cannot know that its two factors are the same variable and obtains $[-1, 1]$. Tighter enclosures reduce the number of subdivisions wherever a curve is plotted or traced.

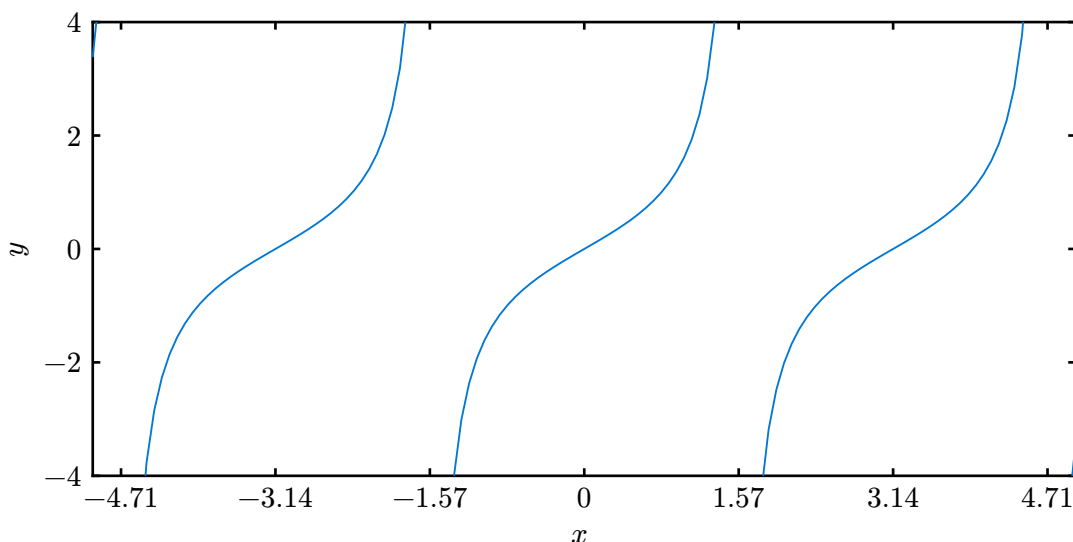
Expanded and factored forms differ in cost; measure both. Interval arithmetic treats each occurrence of a variable independently, so the same curve costs differently depending on how it is written. The hyperbola $(20*y + x) * (20*y - x) = 1$ traces with 232 points and 44 uncertain cells. Expanded to $400*y^2 - x^2 = 1$, the derivative enclosures reduce to the exact $-2x$ and $800y$, and the same window costs 144 points with 4 uncertain. A product form $g * h = 0$, on the other hand, states the union structure explicitly, which is what makes uncertain: "join" the informed choice at the crossings of g and h . The two objectives conflict; each form costs one call to evaluate, and the uncertain count is the criterion.

Use with plotting packages

Chains and rasters are plain Typst data, so no adapters are required. Either package below can draw either figure; the two examples differ only in what they demonstrate.

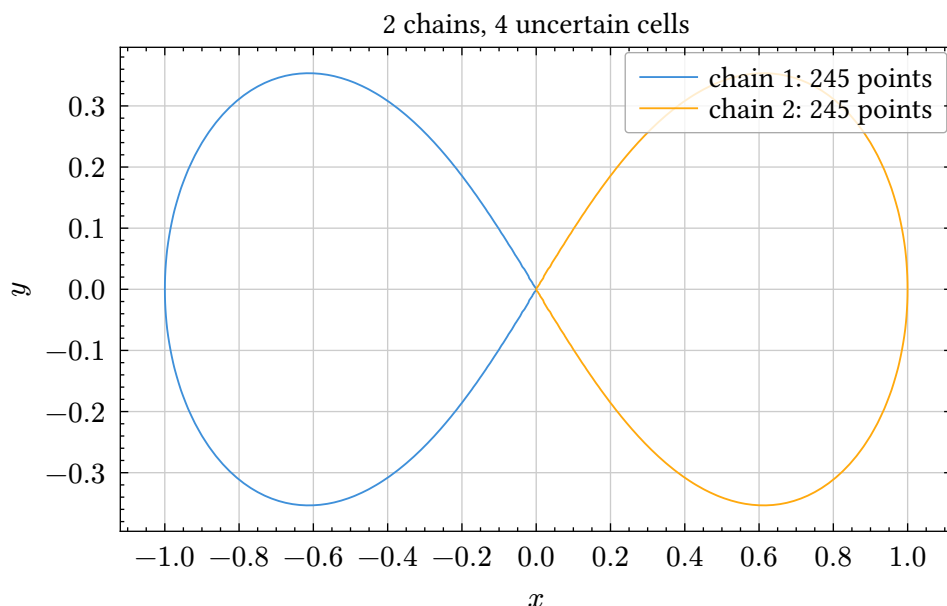
With **cetz-plot**, each chain is one add call. This example uses the axis context: ticks placed at multiples of $\frac{\pi}{2}$ fall exactly on the asymptotes and label where the curve stops:

```
#cetz.canvas({
  cplot.plot(
    size: (13, 6),
    x-min: -5, x-max: 5, y-min: -4, y-max: 4,
    x-tick-step: calc.pi / 2, x-format: "sci",
    y-tick-step: 2,
    x-label:  $x$ , y-label:  $y$ ,
    {
      for chain in tangent.chains {
        cplot.add(chain, style: (stroke: 0.7pt + accent), mark: none)
      }
    },
  ),
})
```



With **lilaq**, each chain is one data series. This example shows the structure of the result: the colour cycle and the legend display exactly what contour returned — the lemniscate decomposes into two lobes that separate at the self-intersection — with the uncertain count in the title:

```
#let lemniscate = ip.contour("(x^2 + y^2)^2 = x^2 - y^2",
  n: 80, x: (-1.2, 1.2), y: (-0.7, 0.7))
#lq.diagram(
  width: 11cm, height: 6.4cm,
  xlabel: $x$, ylabel: $y$,
  title: [#chains-label(lemniscate.chains.len()),
    #lemniscate.uncertain uncertain cells],
  ..lemniscate.chains.enumerate().map((k, chain)) => lq.plot(
    chain.map(p => p.at(0)),
    chain.map(p => p.at(1)),
    mark: none,
    label: [chain #(k + 1): #chain.len() points],
  ),
)
```



API reference

The reference below is generated by the `tidy` package from the doc comments in `lib.typ`, so it cannot drift from the module. Its examples are evaluated during the build.

- `contour()`
- `plot()`
- `rows()`

contour

Trace the curve of a relation as polylines, by subdivision with a guaranteed topology [3].

Returns a dictionary (`chains`, `uncertain`). Each chain is an array of (x, y) pairs in the relation's own coordinates; closed curves repeat their first point, so a chain can be stroked without special-casing.

uncertain counts cells where the curve is singular - a self-intersection, where the gradient vanishes and no subdivision can settle the topology. Zero means the shape is guaranteed correct, not merely plausible.

`plot()` answers a different question - which pixels the curve may touch - and tracing that raster is not a substitute: it follows the outline of the band of lit pixels, giving staircase fragments where this gives one path.

```
#let c = contour("x^2 + y^2 = 1", n: 24,  
x: (-1.5, 1.5), y: (-1.5, 1.5))  
The unit circle: #c.chains.len() chain of  
#c.chains.at(0).len() points,  
#c.uncertain uncertain cells.
```

The unit circle: 1 chain of 85 points, 0 uncertain cells.

Parameters

```
contour(  
  relation: str,  
  n: int,  
  x: array,  
  y: array,  
  refine: int,  
  uncertain: str  
) -> dictionary
```

relation str

The relation whose curve to trace, in the same language `plot()` takes. Only the two sides are read: the curve is where they are equal, so "`f <= 1`" traces the boundary "`f = 1`" and the comparison written is not consulted.

n int

How finely to resolve the curve. The tree is adaptive: this is the resolution the curve is resolved **to**, not a grid walked everywhere - empty regions cost one evaluation each.

Default: 60

x array

The horizontal extent of the plane to trace.

Default: (-10, 10)

y array

The vertical extent of the plane to trace.

Default: (-10, 10)

refine `int`

How far below n 's resolution an unresolved cell may be split before it is given up as uncertain. Each level costs about four times the last, for shrinking returns: a genuine singularity never resolves however deep this goes, so raising it mainly tightens the region around singular points and resolves features finer than n .

Default: `2`

uncertain `str`

What to draw inside the cells counted by uncertain. The curve has values there; what cannot be computed is how the arcs connect, so the choice is the caller's. "avoid" keeps the arcs apart - right when a "crossing" is a near miss. "join" contracts each cluster of uncertain cells to a single junction, giving a true X - right whenever the relation visibly factors, as $\sin(x^2+y^2) = \cos(x*y)$ does into two ellipse families.

Default: `"avoid"`

plot

Plot the region satisfying a relation and return one byte per pixel, row-major from the top - not an image; the document decides what to draw.

The plotter subdivides adaptively with interval arithmetic, so a lit pixel means the relation provably or plausibly holds there and an unlit pixel means it provably does not: coverage errs outward, never inward.

```
#raw(rows(plot("x < 0", size: (6, 6), x:
(-1, 1), y: (-1, 1)), 6)
.map(row => row.map(str).join("
")).join("\n"))
```

```
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
```

Parameters

```
plot(
  relation: str,
  size: array,
  x: array,
  y: array,
  depth: int
) -> bytes
```

relation `str`

The relation, written as text: `"sin(x^2 + y^2) = cos(x*y)"`.

The language is x , y , the constants π , τ and e , decimal numbers (1.5e3 included), $+$ $-$ $*$ $/$ $\%$, parentheses, $^$ with an integer literal exponent, the functions `neg` `abs` `sin` `cos` `tan` `exp` `sqrt` `log` `asin` `acos` `atan` `floor` `ceil` — with `ln`, `arcsin`, `arccos` and `arctan` as alternative spellings — and `min(a, b)` and `max(a, b)`. $\%$ is floored modulo, taking the sign of its divisor as Python's does. The comparison is one of `=` `<` `<=` `>` `>=`, written between the two sides; `==` is accepted for `=`. The dedicated minus sign `-` (U+2212) is accepted wherever `-` is, so a formula copied out of rendered math parses as written. Whitespace is insignificant, but multiplication is never implicit: `2x` is an error, and `x^y` is one too.

`floor`, `ceil` and $\%$ are the only discontinuous operations. `contour()` refuses to trace across a step and counts those cells in uncertain; `plot` needs no continuity and handles them exactly.

Everything is reduced to $f \bmod 0$ before it is plotted, so `"f > 1"` and `"f - 1 > 0"` are the same relation and cost the same.

Prefer x^2 to $x*x$: the plotter encloses $[-1, 1]^2$ as $[0, 1]$, where a product of two intervals can only manage $[-1, 1]$, so the power converges in fewer subdivisions.

size `array`

Width and height in pixels.

Default: `(64, 64)`

x `array`

The horizontal extent of the plane to cover.

Default: `(-10, 10)`

y `array`

The vertical extent of the plane to cover.

Default: `(-10, 10)`

depth `int`

How many times a single undecided pixel may be subdivided before giving up on proving it.

Default: `8`

rows

Split a flat plot into rows, top first.

Parameters

```
rows(  
    pixels: bytes array ,  
    width: int  
) -> array
```

pixels bytes or array

The result of `plot()` (or any flat sequence).

width int

The row width, i.e. the first element of the size the plot was made at.

Bibliography

- [1] J. Tupper, “Reliable Two-Dimensional Graphing Methods for Mathematical Formulae with Two Free Variables,” in *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*, ACM, 2001, pp. 77–86. doi: 10.1145/383259.383267.
- [2] S. Plantinga and G. Vegter, “Isotopic Approximation of Implicit Curves and Surfaces,” in *Proceedings of the 2004 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing (SGP '04)*, ACM, 2004, pp. 245–254. doi: 10.1145/1057432.1057465.
- [3] L. Lin and C. Yap, “Adaptive Isotopic Approximation of Nonsingular Curves: the Parameterizability and Nonlocal Isotopy Approach,” *Discrete & Computational Geometry*, vol. 45, no. 4, pp. 760–795, 2011, doi: 10.1007/s00454-011-9345-9.
- [4] IEEE, “IEEE Standard for Interval Arithmetic,” IEEE, IEEE Std 1788–2015, 2015. doi: 10.1109/IEEESTD.2015.7140721.
- [5] J. M. Snyder, “Interval Analysis for Computer Graphics,” *ACM SIGGRAPH Computer Graphics*, vol. 26, no. 2, pp. 121–130, 1992, doi: 10.1145/142920.134024.
- [6] D. Richardson, “Some Undecidable Problems Involving Elementary Functions of a Real Variable,” *The Journal of Symbolic Logic*, vol. 33, no. 4, pp. 514–520, 1968, doi: 10.2307/2271358.